ALUMNO:

María Fernanda Fuentes Abascal

DOCENTE:

Octavio Sanchez Delgado

ACTIVIDAD:

Actividad de Evaluación 2da parte

MATERIA:

Seguridad en el desarrollo de aplicaciones

FECHA:

29/03/2026

Evaluación de Seguridad en OWASP Juice Shop

1. Introducción

La seguridad en las aplicaciones web es un aspecto fundamental dentro del desarrollo de software, ya que permite proteger la información de los usuarios y evitar accesos no autorizados a los sistemas. Muchas aplicaciones presentan vulnerabilidades que pueden ser explotadas por atacantes para comprometer datos, modificar información o ejecutar código malicioso.

En esta práctica se hizo un análisis de seguridad utilizando OWASP Juice Shop, una aplicación web vulnerable diseñada específicamente para el aprendizaje y práctica de pruebas de seguridad. El objetivo fue identificar vulnerabilidades mediante escaneo automatizado y pruebas manuales, enfocándose principalmente en ataques de SQL Injection (SQLi) y Cross-Site Scripting (XSS).

2. Resultados del escaneo automatizado

Para realizar el escaneo automatizado se utilizó una herramienta de análisis de seguridad (como OWASP ZAP) sobre la aplicación ejecutada en la dirección:

http://localhost:3008

Durante el análisis se detectaron diversas vulnerabilidades relacionadas con la seguridad de la aplicación. Entre los hallazgos más relevantes se encuentran:

- Falta de algunos encabezados de seguridad HTTP.
- Posibles puntos vulnerables a ataques de Cross-Site Scripting.
- Exposición de información en ciertas respuestas del servidor.
- Configuración insegura en algunos elementos de autenticación.

Estas vulnerabilidades indican que la aplicación no implementa completamente medidas de protección para validar y sanitizar las entradas de los usuarios.

3. Resultados de las pruebas manuales

Durante las pruebas manuales se realizaron diferentes intentos para identificar vulnerabilidades comunes en aplicaciones web.

3.1 Prueba de SQL Injection

La prueba de SQL Injection se realizó en el formulario de autenticación (login) de la aplicación.

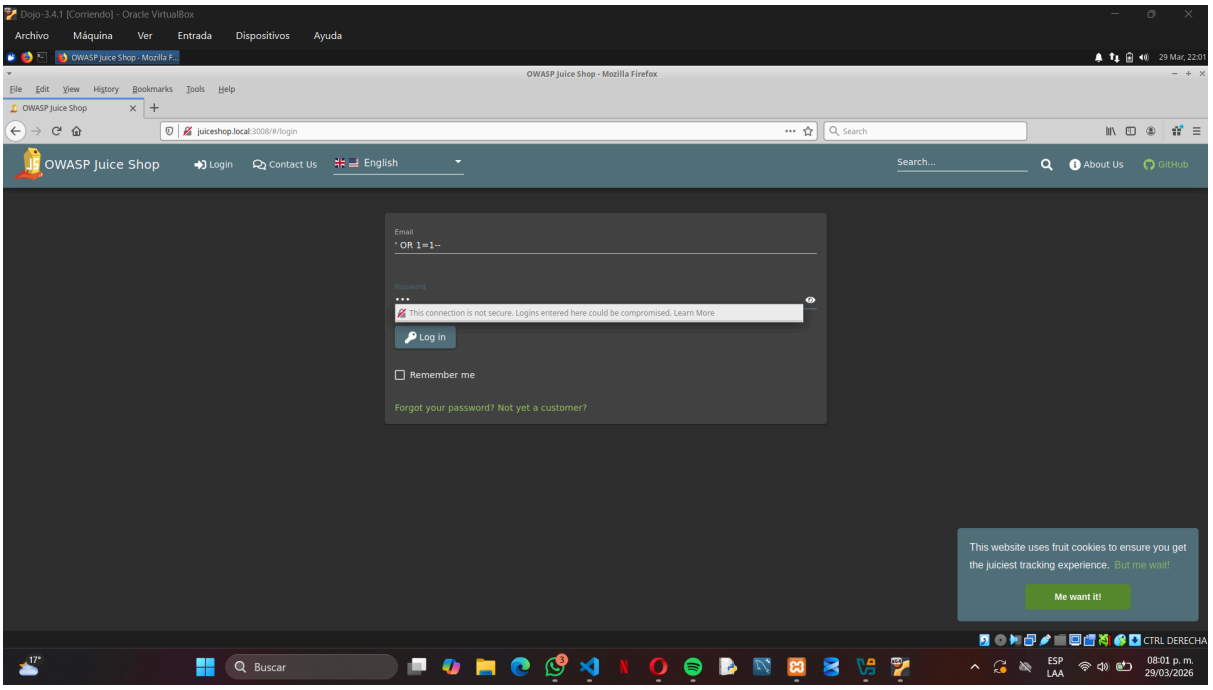
En el campo de correo electrónico se introdujo el siguiente código:

' OR 1=1--

Mientras que en el campo de contraseña se ingresó un valor cualquiera.

El sistema permitió el acceso al sistema aun sin introducir credenciales válidas, lo que indica que la consulta SQL del sistema puede ser manipulada mediante la entrada del usuario.

Esto ocurre porque la condición OR 1=1 siempre se evalúa como verdadera, permitiendo que el sistema devuelva registros de la base de datos sin validar correctamente las credenciales.



3.2 Prueba de Cross-Site Scripting (XSS)

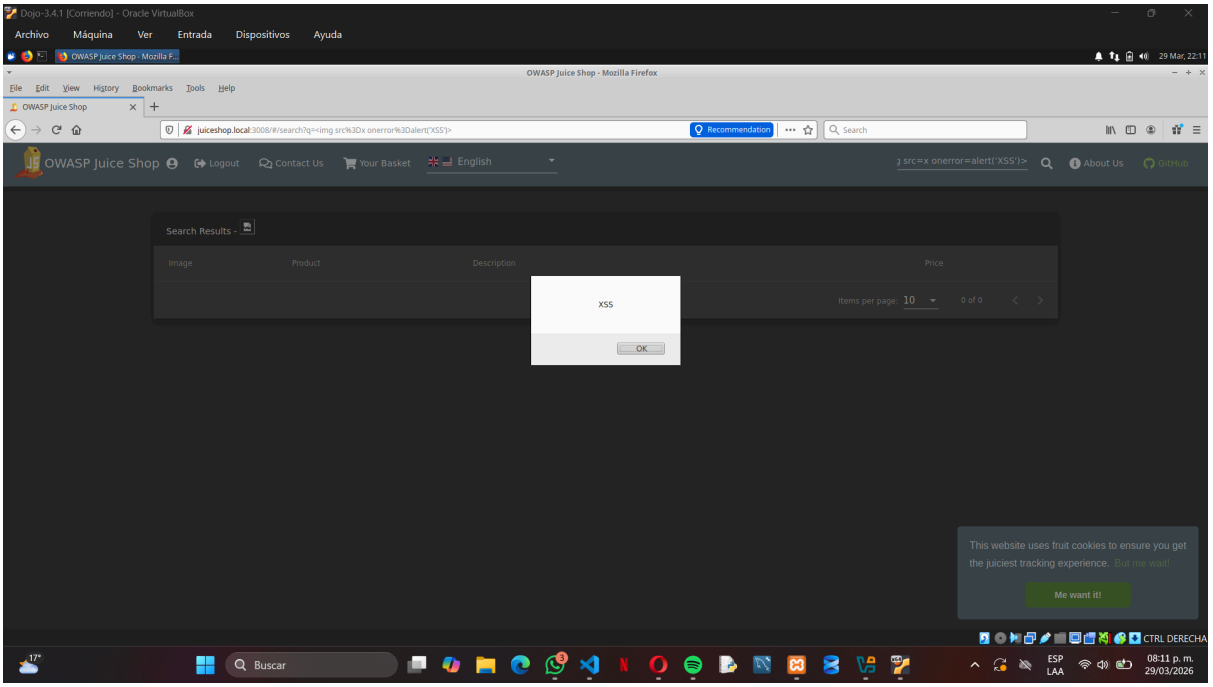
Para detectar vulnerabilidades de tipo XSS se introdujo código JavaScript dentro del buscador de la aplicación.

El siguiente payload fue utilizado:

<img src=x onerror=alert('XSS')>

Este tipo de código intenta ejecutar un script dentro del navegador del usuario. Si la aplicación no valida o sanitiza correctamente la entrada, el navegador ejecutará el código.

Este comportamiento demuestra que el sistema puede permitir la ejecución de scripts maliciosos, lo que representa un riesgo para los usuarios, ya que un atacante podría robar sesiones, modificar contenido o redirigir a páginas maliciosas.



4. Grado de alerta de las vulnerabilidades

Vulnerabilidad	Nivel de alerta	Descripción
SQL Injection	**Alta**	Permite manipular consultas a la base de datos y acceder al sistema sin credenciales válidas.
Cross-Site Scripting (XSS)	**Alta**	Permite ejecutar código JavaScript en el navegador del usuario.
Encabezados de seguridad faltantes	**Media**	Puede facilitar ataques adicionales al navegador.
Exposición de información	**Baja**	Puede revelar datos del sistema que faciliten otros ataques.

5. Propuesta de solución

Para reducir o eliminar estas vulnerabilidades es necesario implementar diversas medidas de seguridad en el desarrollo de la aplicación.

Para prevenir SQL Injection:

- Utilizar consultas preparadas (Prepared Statements).
- Validar y filtrar todas las entradas del usuario.
- Implementar frameworks o ORM que gestionen las consultas a la base de datos de forma segura.

Para prevenir Cross-Site Scripting (XSS):

- Sanitizar las entradas del usuario antes de mostrarlas en la página.
- Escapar caracteres HTML especiales.
- Implementar políticas de seguridad de contenido (Content Security Policy).

Para mejorar la seguridad general:

- Configurar correctamente encabezados HTTP de seguridad.
- Implementar autenticación segura.
- Limitar la exposición de información del sistema.

6. Conclusiones

El análisis realizado en la aplicación OWASP Juice Shop permitió identificar diversas vulnerabilidades de seguridad comunes en aplicaciones web. Las pruebas manuales demostraron que es posible explotar vulnerabilidades como SQL Injection y Cross-Site Scripting cuando no se aplican mecanismos adecuados de validación y sanitización de datos.

Este ejercicio permitió comprender la importancia de aplicar buenas prácticas de seguridad durante el desarrollo de software, así como utilizar herramientas de análisis para detectar posibles fallas antes de que una aplicación sea desplegada en un entorno real.

Además, se concluye que la seguridad debe ser considerada desde las primeras etapas del desarrollo para proteger la información de los usuarios y evitar posibles ataques.

7. Bibliografía

OWASP Foundation. (2023). OWASP Top 10 Web Application Security Risks. https://owasp.org

OWASP Juice Shop Project. (2024). OWASP Juice Shop Documentation. https://owasp.org/www-project-juice-shop/

Stuttard, D., & Pinto, M. (2011). The Web Application Hacker's Handbook. Wiley Publishing.